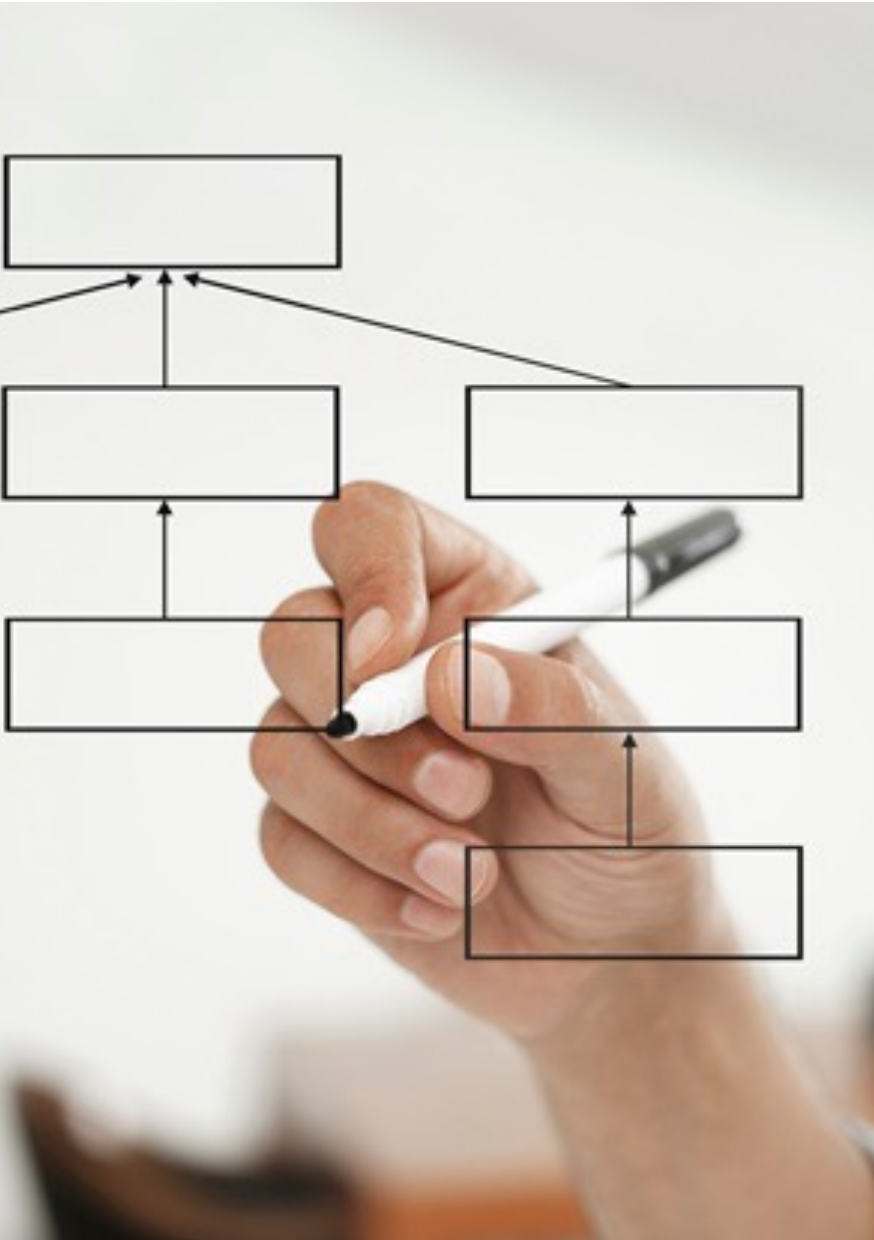




Estrutura de Dados I

CÁSSIO CAPUCHO PEÇANHA - 05

Busca em Array

Definição

- Ato de procurar por um elemento em um conjunto de dados
 - Recuperação de dados armazenados em um repositório ou base de dados
- A operação de busca visa responder se um determinado valor está ou não presente em um conjunto de elementos
 - Por exemplo, em um array

Definição

- Baseada em uma chave
 - A chave de busca é o **campo** do item utilizado para comparação
 - Valor armazenado em um array de inteiros
 - Campo de uma struct
 - etc
 - É por meio dela que sabemos se dado elemento é o que buscamos
 - No caso do item estar presente no conjunto de elementos, seus dados são retornados para o usuário

Definição

- Existem vários tipos de busca
- Sua utilização depende de como são estes dados
 - Os dados estão estruturados ?
 - Array, lista, árvore, etc.?
 - Existe também a busca em dados não estruturados
 - Os dados estão ordenados?
 - Existem valores duplicados?

Definição

- Tipos de busca abordados
 - Dados armazenados em um array
 - Dados ordenados ou não
- Métodos
 - Busca Sequencial ou Linear
 - Busca Sequencial Ordenada
 - Busca Binária

	0	1	2	3	4	5	6
V	23	4	67	-8	54	90	21

Busca Sequencial ou Linear

- Estratégia de busca mais simples que existe
 - Basicamente, esse algoritmo percorre o array que contém os dados desde a sua primeira posição até a última
 - Assume que os dados não estão ordenados, por isso a necessidade de percorrer o array do seu início até o seu fim

Busca Sequencial ou Linear

■ Funcionamento

- Para cada posição do array, o algoritmo compara se a posição atual do array é igual ao valor buscado.
 - Se os valores forem iguais, a busca termina
 - caso contrário, a busca continua com a próxima posição do array

Busca Sequencial ou Linear

■ Algoritmo

```
13 |
14 | □ int buscaLinear(int *V, int N, int elem){
15 |     int i;
16 |     □ for(i = 0; i<N; i++){
17 |         if(elem == V[i])
18 |             return i; //elemento encontrado
19 |     }
20 |     return -1; //elemento não encontrado
21 | }
```

Busca Sequencial ou Linear - Exemplo

	0	1	2	3	4	5	6
V	23	4	67	-8	54	90	21

elem **54** Elemento procurado

	0	1	2	3	4	5	6	
i=0	23	4	67	-8	54	90	21	Valor diferente: continua a busca
i=1	23	4	67	-8	54	90	21	Valor diferente: continua a busca
i=2	23	4	67	-8	54	90	21	Valor diferente: continua a busca
i=3	23	4	67	-8	54	90	21	Valor diferente: continua a busca
i=4	23	4	67	-8	54	90	21	Valor igual: termina a busca

Busca Sequencial ou Linear

- Complexidade
 - Considerando um array com N elementos
 - $O(1)$, melhor caso: o elemento é o primeiro do array
 - $O(N)$, pior caso: o elemento é o último do array ou não existe
 - $O(N/2)$, caso médio

Busca Sequencial Ordenada

- Procurar por um determinado valor em um array desordenado é uma tarefa bastante cara
- Como melhorar isso?
 - Organizando o array segundo alguma ordem, isto é, devemos ordenar o array
 - Isso facilita a tarefa de busca

Busca Sequencial Ordenada

- Funcionamento

- Assume que os dados estão ordenados
- Se o elemento procurado for **menor** do que o valor em uma determinada posição do array, temos a certeza de que ele não estará no restante do array
 - Isso evita a necessidade de percorrer o array do seu início até o seu fim

Busca Sequencial Ordenada

■ Algoritmo

```
14
15 □ int buscaOrdenada(int *V, int N, int elem){
16   |   int i;
17   □   for(i = 0; i<N; i++){
18       |       if(elem == V[i])
19           |       return i; //elemento encontrado
20       |       else
21           |       if(elem < V[i])
22               |       return -1; //para a busca
23       |   }
24   |   return -1; //elemento não encontrado
25   | }
```

Busca Sequencial Ordenada - Exemplo

	0	1	2	3	4	5	6
V	-8	4	21	23	54	67	90

elem 34 Elemento procurado

	0	1	2	3	4	5	6
i=0	-8	4	21	23	54	67	90

 Valor diferente: continua a busca

i=1	-8	4	21	23	54	67	90
-----	----	---	----	----	----	----	----

 Valor diferente: continua a busca

i=2	-8	4	21	23	54	67	90
-----	----	---	----	----	----	----	----

 Valor diferente: continua a busca

i=3	-8	4	21	23	54	67	90
-----	----	---	----	----	----	----	----

 Valor diferente: continua a busca

i=4	-8	4	21	23	54	67	90
-----	----	---	----	----	----	----	----

 Valor é maior: elemento não existe

Busca Sequencial Ordenada

- Desvantagens

- Ordenar um array também tem um custo

- Esse custo é superior ao custo da busca sequencial no seu pior caso

- Se for para fazer a busca de um único elemento, não compensa ordenar o array

- Porém, se mais de um elemento for recuperado do array, o esforço de ordenar o array pode compensar

Busca Binária

- Fazer a busca em um array ordenado representa um ganho de tempo
 - Podemos terminar a busca mais cedo se o elemento procurado for menor que o valor da posição atual do array

Busca Binária

- A Busca Sequencial Ordenada é uma estratégia de busca extremamente simples
 - Ela percorre todo o array linearmente
 - Não utiliza adequadamente a ordenação dos dados
- Uma estratégia de busca mais sofisticada é a Busca Binária
 - Muito mais eficiente do que a Busca Sequencial Ordenada

Busca Binária

- Funcionamento

- É uma estratégia baseada na idéia de *dividir para conquistar*
- A cada passo, esse algoritmo analisa o valor do meio do array
 - Caso esse valor seja igual ao elemento procurado, a busca termina
 - Caso contrário, a busca continua na metade do array que condiz com o valor procurado

Busca Binária

■ Algoritmo

```
27
28 □ int buscaBinaria(int *V, int N, int elem) {
29     int i, inicio, meio, final;
30     inicio = 0;
31     final = N-1;
32     □ while(inicio <= final){
33         meio = (inicio + final)/2;
34         if(elem < V[meio])
35             final = meio-1; //busca na metade da esquerda
36         else
37             if(elem > V[meio])
38                 inicio = meio+1; //busca na metade da direita
39             else
40                 return meio;
41     }
42     return -1; //elemento não encontrado
43 }
```

Busca Binária - Exemplo

	0	1	2	3	4	5	6	7	8	9
V	-8	-5	1	4	14	21	23	54	67	90

elem **4** Elemento procurado

	0	1	2	3	4	5	6	7	8	9
meio=4	-8	-5	1	4	14	21	23	54	67	90
Valor é menor: buscar no início										

	0	1	2	3	4	5	6	7	8	9
meio=1	-8	-5	1	4	14	21	23	54	67	90
Valor é maior: buscar no final										

	0	1	2	3	4	5	6	7	8	9
meio=2	-8	-5	1	4	14	21	23	54	67	90
Valor é maior: buscar no final										

	0	1	2	3	4	5	6	7	8	9
meio=3	-8	-5	1	4	14	21	23	54	67	90
Valor é igual: terminar a busca										

Busca Binária

- Complexidade

- Considerando um array com N elementos, o tempo de execução é:
 - $O(1)$, melhor caso: o elemento procurado está no meio do array;
 - $O(\log_2 N)$, pior caso: o elemento não existe;
 - $O(\log_2 N)$, caso médio.

Busca Binária

- Complexidade

- Para se ter uma idéia da sua vantagem, em um array contendo **$N = 1000$** elementos, no pior caso
 - A Busca Sequencial irá executar **1000 comparações**
 - A Busca Binária irá executar apenas **10 comparações**

Busca em array de struct

- A busca em um array de inteiros é uma tarefa simples
 - Na prática, trabalhamos com dados um pouco mais complexos, como estruturas
 - Mais dados para manipular

```
11 struct aluno{  
12     int matricula;  
13     char nome[30];  
14     float n1,n2,n3;  
15 };
```

Busca em array de struct

- Como fazer a busca quando o que temos é um array de struct?

```
struct aluno V[6];
```

matricula; nome[30]; n1,n2,n3;	matricula; nome[30]; n1,n2,n3;	matricula; nome[30]; n1,n2,n3;	matricula; nome[30]; n1,n2,n3;	matricula; nome[30]; n1,n2,n3;	matricula; nome[30]; n1,n2,n3;
V[0]	v[1]	v[2]	v[3]	v[4]	v[5]

Busca em array de struct

Relembrando

- A busca é baseada em uma chave
 - A chave de busca é o **campo** do item utilizado para comparação
 - Valor armazenado em um array de inteiros
 - **Campo de uma struct**
 - etc
 - É por meio dela que sabemos se dado elemento é o que buscamos
 - No caso do item estar presente no conjunto de elementos, seus dados são retornados para o usuário

Busca em array de struct

- Ou seja, devemos modificar o algoritmo para que a comparação das chaves seja feita utilizando um determinado campo da **struct**

- Exemplo

- Vamos modificar a **busca linear**
 - Essa modificação vale para os outros tipos de busca

```
13 |
14 | int buscaLinear(int *V, int N, int elem) {
15 |     int i;
16 |     for(i = 0; i < N; i++) {
17 |         if(elem == V[i])
18 |             return i; // elemento encontrado
19 |     }
20 |     return -1; // elemento não encontrado
21 | }
```

Busca em array de struct

- Duas novas buscas
 - Busca por matricula
 - Busca por nome

```
28 int buscaLinearMatricula(struct aluno *V, int N, int elem){
29     int i;
30     for(i = 0; i<N; i++){
31         if(elem == V[i].matricula)
32             return i;//elemento encontrado
33     }
34     return -1;//elemento não encontrado
35 }
36
37
38 int buscaLinearNome(struct aluno *V, int N, char* elem){
39     int i;
40     for(i = 0; i<N; i++){
41         if(strcmp(elem,V[i].nome)==0)
42             return i;//elemento encontrado
43     }
44     return -1;//elemento não encontrado
45 }
```